

1. FizzBuzz Variation

```
public class FizzBuzz {
    public static void main(String[] args) {
        int n = 30;

        for (int i = 1; i <= n; i++) {

            if (i % 3 == 0 && i % 5 == 0)
                System.out.println("FizzBuzz");
            else if (i % 3 == 0)
                System.out.println("Fizz");
            else if (i % 5 == 0)
                System.out.println("Buzz");
            else
                System.out.println(i);
        }
    }
}
```

2. Prime Number Checker

```
public class PrimeNumberCheckher {

    public static boolean isPrime(int no) {

        if (no <= 1) {
            return false;
        }

        for (int i = 2; i < no; i++) {
            if (no % i == 0) {
                return false;
            }
        }
        return true;
    }

    public static void FindFactors(int no) {

        for (int i = 1; i <= no; i++) {
```

```

        if (no % i == 0)
            System.out.println(i);
    }
}

public static void main(String[] args) {
    int no = 12;

    if (isPrime(no)) {
        System.out.println("Prime");
    } else {
        System.out.println("Not Prime");
        FindFactors(no);
    }
}
}

```

3. Sum of Digits Until Single Digit

```

public class SumOfDigit {
    public static int Result2() {
        int no = 12345;

        while (no > 9) {
            int sum = 0;

            while (no != 0) {
                int rem = no % 10;
                sum += rem;
                no = no / 10;
            }
            no = sum;
        }
        return no;
    }

    public static void main(String[] args) {
        int no = 12345;
        int singleDigit = 1 + (no - 1) % 9;
        System.out.println(singleDigit);
    }
}

```

```
}
```

4. Reverse a Number and Check Palindrome

```
public class CheckPalindrome {

    public static boolean checkPalindrome(int no) {
        int reverse = 0;
        int temp = no;

        while (no != 0) {
            int rem = no % 10;
            reverse = reverse * 10 + rem;
            no = no / 10;
        }

        System.out.println("Reverse : " + reverse);

        if (reverse == temp) {
            return true;
        } else {
            return false;
        }
    }

    public static void main(String[] args) {
        if (checkPalindrome(1221))
            System.out.println("Palindrome Number");
        else
            System.out.println("Not Palindrome");
    }
}
```

5. Find GCD and LCM of Two Numbers.

```
public class GCD_LCM {

    public static int findGCD(int no1, int no2) {
        int result = 0;
```

```

        for (int i = 1; i <= no1; i++) {
            if (no1 % i == 0 && no2 % i == 0)
                result = i;
        }
        return result;
    }

    public static int findLCM(int no1, int no2) {
        int result = (no1 * no2) / findGCD(no1, no2);
        return result;
    }

    public static void main(String[] args) {
        int no1 = 12;
        int no2 = 18;

        System.out.println(findGCD(no1, no2));
        System.out.println(findLCM(no1, no2));
    }
}

```

6. Second Largest Element in Array

```

import java.util.ArrayList;
import java.util.Arrays;

public class SecondLargest {

    public static int FindSecondLargest(ArrayList<Integer> list) {
        int largest = list.get(0);
        int sLargest = -1;

        for (int i : list) {
            if (i > largest) {
                sLargest = largest;
                largest = i;
            } else if (i < largest && i > sLargest)
                sLargest = i;
        }
    }
}

```

```

        return sLargest;
    }

    public static void main(String[] args) {
        ArrayList<Integer> list = new ArrayList<>(Arrays.asList(10, 20, 30, 40, 45, 50));
        System.out.println(FindSecondLargest(list));
    }
}

```

7. Remove Duplicates from Array

```

public class RemoveDuplicatedFromArray {

    public static void RemoveDuplicate(int[] a) {
        HashSet<Integer> set = new HashSet<>();

        for (int i = 0; i < a.length; i++) {
            set.add(a[i]);
        }

        for (int i : set) {
            System.out.println(i);
        }
    }

    public static void main(String[] args) {
        int[] a = { 1, 1, 2, 1, 2, 2, 4, 5, 6, 7 };
        RemoveDuplicate(a);
    }
}

```

8. Rotate Array by k Positions

```

public class RotateArrayByK {

    static void Rotate1(int[] a, int k) {
        int n = a.length;
        ArrayList<Integer> list = new ArrayList<>();
    }
}

```

```

    k = k % n;

    for (int i = n - k; i < n; i++) {
        list.add(a[i]);
    }

    for (int i = 0; i < n - k; i++) {
        list.add(a[i]);
    }

    for (int i : list) {
        System.out.println(i);
    }
}

static void reverse(int[] a, int start, int end) {
    int i = start, j = end;

    while (i <= j) {
        int temp = a[i];
        a[i] = a[j];
        a[j] = temp;

        i++;
        j--;
    }
}

static void Rotate2(int[] a, int k) {
    reverse(a, 0, a.length - k - 1);
    reverse(a, a.length - k, a.length - 1);
    reverse(a, 0, a.length - 1);

    for (int i : a) {
        System.out.print(i + " ");
    }
}

public static void main(String[] args) {
    int a[] = { 10, 20, 30, 40, 50, 60 };
    int k = 3;

```

```
        Rotate2(a, k);
    }
}
```

9. Find Missing Number in Sequence.

```
public class FindMissingNumber {

    static int FindMissingNumber(int[] a, int n) {

        int sum = 0;
        int TotalSum = n * (n + 1) / 2;

        for (int i = 0; i < n - 1; i++) {
            sum += a[i];
        }

        return TotalSum - sum;
    }

    public static void main(String[] args) {
        int[] a = { 1, 2, 3, 5 };
        int n = 5;
        System.out.println(FindMissingNumber(a, n));
    }
}
```

10. Check if Array is Sorted

```
public class ArraySortedOrNot {

    static boolean checkAscending(int[] a) {

        for (int i = 0; i < a.length - 1; i++) {

            if (a[i] != a[i + 1] && a[i] > a[i + 1])
                return false;
        }
        return true;
    }
}
```

```

    }

    static boolean checkDescending(int[] a) {
        for (int i = 0; i < a.length - 1; i++) {
            if (a[i] != a[i + 1] && a[i] < a[i + 1])
                return false;
        }
        return true;
    }

    public static void main(String[] args) {
        int[] a = { 1, 1, 2, 3, 4, 5 };
        int[] b = { 5, 5, 4, 3, 2, 1 };
        int[] c = { 1, 2, 5, 2, 1, 1 };

        if (checkAscending(c)) {
            System.out.println("Ascending Array");
        } else if (checkDescending(c)) {
            System.out.println("Descending Array");
        } else {
            System.out.println("Unsorted Array");
        }
    }
}

```

11. Anagram Checker

```

import java.lang.reflect.Array;
import java.util.Arrays;
import java.util.HashMap;
import java.util.LinkedHashMap;

public class AnagramChecker {

    static boolean checkAnagram(String str1, String str2) {

        if (str1.length() != str2.length())
            return false;

        str1 = str1.toLowerCase();
        str2 = str2.toLowerCase();
    }
}

```



```

char[] arr1 = str1.toCharArray();
char[] arr2 = str2.toCharArray();

Arrays.sort(arr1);
Arrays.sort(arr2);

if (Arrays.equals(arr1, arr2)) {
    return true;
} else {
    return false;
}
}

static boolean checkAnagram2(String str1, String str2) {

    HashMap<Character, Integer> map = new LinkedHashMap<>();

    if (str1.length() != str2.length())
        return false;

    for (int i = 0; i < str1.length(); i++) {
        char ch = str1.charAt(i);
        map.put(ch, map.getOrDefault(ch, 0) + 1);
    }

    for (int i = 0; i < str2.length(); i++) {
        char ch = str2.charAt(i);
        map.put(ch, map.getOrDefault(ch, 0) - 1);

        if (map.get(ch) == 0)
            map.remove(ch);
    }

    if (map.isEmpty()) {
        return true;
    }
    return false;
}

public static void main(String[] args) {
    String str1 = "cat";
    String str2 = "bat";
    System.out.println(checkAnagram2("listen", "silent")); // true
}

```

```

        System.out.println(checkAnagram2("hello", "ollhe")); // true
        System.out.println(checkAnagram2("hello", "world")); // false
        System.out.println(checkAnagram2("cat", "bat")); // false

        // (str1, str2);
    }
}

```

12. Count Vowels and Consonants

```

public class VowelConsonent {

    static void check(String str) {
        int vcount = 0;
        int ccount = 0;

        str = str.toLowerCase();

        for (int i = 0; i < str.length(); i++) {
            char ch = str.charAt(i);

            if (ch >= 'a' && ch <= 'z') {
                if (ch == 'a' || ch == 'e' || ch == 'i' || ch == 'o' || ch == 'u')
                    vcount++;
                else
                    ccount++;
            }
        }

        System.out.println("Vowel Count " + vcount);
        System.out.println("Consonent Count " + ccount);
    }

    public static void main(String[] args) {
        String str = "Prathmesh 123!";
        check(str);
    }
}

```

13. Longest Word in a Sentence

```
import java.util.HashMap;
import java.util.Map;

public class WordFrequency {

    public static void FrequencyCount(String str) {
        str = str.toLowerCase();
        String[] arr = str.split(" ");

        HashMap<String, Integer> map = new HashMap<>();

        for (int i = 0; i < arr.length; i++) {
            map.put(arr[i], map.getDefault(arr[i], 0) + 1);
        }
        map.forEach((k, v) -> System.out.println(k + " " + v));
    }

    public static void main(String[] args) {
        String str = "Java is fun, Java is powerful.";
        FrequencyCount(str);
    }
}
```

14. Toggle Case of Each Character

```
import java.util.Scanner;

public class ToggleCase {

    static String ToggleCase(String str) {

        String result = "";

        for (int i = 0; i < str.length(); i++) {
            char ch = str.charAt(i);

            if (Character.isUpperCase(ch)) {
                result += Character.toLowerCase(ch);
            }
        }
    }
}
```

```

        } else if (Character.isLowerCase(ch)) {
            result += Character.toUpperCase(ch);
        } else {
            result += ch;
        }
    }
    return result;
}

static String ToggleCase1(String str) {
    StringBuffer sb = new StringBuffer();

    for (int i = 0; i < str.length(); i++) {
        char ch = str.charAt(i);
        if (ch >= 'A' && ch <= 'Z')
            sb.append((char) (ch + 32));
        else if (ch >= 'a' && ch <= 'z')
            sb.append((char) (ch - 32));
        else
            sb.append(ch);
    }
    return sb.toString();
}

public static void main(String[] args) {
    String str = "JavaPrograming123?!";
    System.out.println(ToggleCase(str));
    System.out.println(ToggleCase1(str));
}
}

```

15. Check Pangram

```
import java.util.HashSet;
```

```
public class CheckPangram {
```

```
    static boolean checkPangram(String str) {
```

```
        str = str.toLowerCase();
```

```
        HashSet<Character> set = new HashSet<>();
```

```

        for (int i = 0; i < str.length(); i++) {
            char ch = str.charAt(i);

            if (ch >= 'a' && ch <= 'z') {
                set.add(ch);
            }
        }

        System.out.println(set.size());

        if (set.size() == 26) {
            return true;
        } else {
            return false;
        }
    }
}

public static void main(String[] args) {
    String str = "The quick brown fox jumps over the lazy dogk";

    System.out.println(checkPangram(str));
}
}

```

16. Armstrong Number Checker

```

public class ArmstrongNumber {

    static boolean checkArmstrong(int no) {
        int temp = no;
        int sum = 0;

        while (no != 0) {
            int r = no % 10;
            sum = sum + (r * r * r);
            no = no / 10;
        }

        if (temp == sum)
            return true;
    }
}

```

```

        else
            return false;
    }

    public static void main(String[] args) {
        if (checkAmstrong(153))
            System.out.println("Amstrong Number");
        else
            System.out.println("Not Amstrong Number");
    }
}

```

17 . Generate Fibonacci Series

```

public class FibonacchiSeries {

    public static void printSeries(int n) {

        int first = 0, second = 1;
        int sum = 0;

        System.out.print(first + " ");
        System.out.print(second + " ");

        for (int i = 1; i < n; i++) {
            sum = first + second;
            System.out.print(sum + " ");
            first = second;
            second = sum;
        }
    }

    public static void main(String[] args) {
        printSeries(10);
    }
}

```

18. Find All Perfect Numbers up to N

```

public class isPerfectNumber {

    public static boolean isPerfectNumber(int no) {
        int sum = 0;

        for (int i = 1; i < no; i++) {
            if (no % i == 0)
                sum += i;
        }

        if (sum == no) {
            return true;
        }

        return false;
    }

    public static void main(String[] args) {
        int n = 1000;

        for (int i = 1; i <= n; i++) {
            if (isPerfectNumber(i))
                System.out.print(i + " ");
        }
    }
}

```

19. Convert Decimal to Binary and Vice Versa.

```

public class BinaryToDecimal {

    public static void DecimalToBinary(int no) {
        StringBuffer sb = new StringBuffer();

        while (no != 0) {
            int rem = no % 2;
            sb.append(rem);
            no = no / 2;
        }
    }
}

```

```

        sb.reverse();
        System.out.println(sb.toString());
    }

    public static void BinaryToDecimal(String str) {

        int sum = 0;

        for (int i = str.length() - 1; i >= 0; i--) {
            int no = str.charAt(i) - '0';

            sum += no * Math.pow(2, str.length() - 1 - i);
        }

        System.out.println(sum);

    }

    public static void main(String[] args) {
        // DecimalToBinary(8);
        BinaryToDecimal("0001");
    }
}

```

20. Sum of Prime Numbers in a Range

```

public class SumOfPrimeNumber {

    public static boolean isPrime(int no) {

        if (no <= 1)
            return false;

        for (int i = 2; i < no; i++) {
            if (no % i == 0)
                return false;
        }

        return true;
    }
}

```



```

    }

    public static void main(String[] args) {
        int n = 20;
        int sum = 0;

        for (int i = 10; i <= n; i++) {
            if (isPrime(i)) {
                sum += i;
            }
        }

        System.out.println(sum);
    }
}

```

21. Pascal's Triangle Generator

```

public class PascalsTriangle {

    public static void PrintTriangle(ArrayList<ArrayList<Integer>> triangle) {
        for (ArrayList<Integer> row : triangle) {
            for (int num : row) {
                System.out.print(num + " ");
            }
            System.out.println();
        }
    }

    public static void findTriangle(int rows) {
        ArrayList<ArrayList<Integer>> triangle = new ArrayList<>();
        ArrayList<Integer> pre = null;
        ArrayList<Integer> newList = null;

        for (int i = 0; i < rows; i++) {

            newList = new ArrayList<>();

            for (int j = 0; j <= i; j++) {

```

```

        if (j == 0 || j == i)
            newList.add(1);
        else
            newList.add(pre.get(j - 1) + pre.get(j));
    }

    pre = newList;
    triangle.add(newList);
}

PrintTriangle(triangle);
}

public static void main(String[] args) {
    int n = 5;
    findTriangle(n);
}
}

```

22. Find Leader Elements in Array

```

import java.util.ArrayList;

public class LeadersInArray {

    public static void ReverseArray(ArrayList<Integer> list) {
        int i = 0, j = list.size() - 1;

        while (i <= j) {
            int temp = list.get(i);
            list.set(i, list.get(j));
            list.set(j, temp);

            i++;
            j--;
        }
    }

    public static void LeadersInArray(int[] a) {
        ArrayList<Integer> list = new ArrayList<>();
    }
}

```

```

int leader = a[a.length - 1];

list.add(leader);

for (int i = a.length - 2; i >= 0; i--) {
    if (a[i] > leader) {
        leader = a[i];
        list.add(a[i]);
    }
}

ReverseArray(list);

System.out.println(list);

}

public static void main(String[] args) {
    int[] a = { 16, 17, 4, 3, 5, 2 };
    LeadersInArray(a);
}
}

```

23. Matrix Transpose

```

public class MatrixTranspose {

    public static void PrintMatrix(ArrayList<ArrayList<Integer>> mArrayList) {
        int row = mArrayList.size();
        int column = mArrayList.get(0).size();

        for (int i = 0; i < column; i++) {
            for (int j = 0; j < row; j++) {
                System.out.print(mArrayList.get(i).get(j) + " ");
            }
            System.out.println();
        }

    }

    public static void MatrixTransposeDemo(ArrayList<ArrayList<Integer>> list) {

```

```

int rows = list.size();
int column = list.get(0).size();

System.out.println("Before Transpose ");
PrintMatrix(list);

// Transpose
ArrayList<ArrayList<Integer>> Transpose = new ArrayList<>();

for (int i = 0; i < rows; i++) {

    ArrayList<Integer> newList = new ArrayList<>();

    for (int j = 0; j < column; j++) {
        newList.add(list.get(j).get(i));
    }
    Transpose.add(newList);
}

System.out.println("After Transpose");
PrintMatrix(Transpose);

}

public static void main(String[] args) {
    // take list input from usera

    ArrayList<ArrayList<Integer>> matrix = new ArrayList<>();

    ArrayList<Integer> row1 = new ArrayList<>();
    row1.add(1);
    row1.add(2);
    row1.add(3);
    ArrayList<Integer> row2 = new ArrayList<>();
    row2.add(4);
    row2.add(5);
    row2.add(6);
    ArrayList<Integer> row3 = new ArrayList<>();
    row3.add(7);
    row3.add(8);
    row3.add(9);

    matrix.add(row1);

```

```

        matrix.add(row2);
        matrix.add(row3);

        MatrixTransposeDemo(matrix);
    }
}

```

24. Count Frequency of Each Word in a Sentence

```

import java.util.HashMap;
import java.util.Map;

public class WordFrequency {

    public static void FrequencyCount(String str) {
        str = str.toLowerCase();
        String[] arr = str.split(" ");

        HashMap<String, Integer> map = new HashMap<>();

        for (int i = 0; i < arr.length; i++) {
            map.put(arr[i], map.getOrDefault(arr[i], 0) + 1);
        }
        map.forEach((k, v) -> System.out.println(k + " " + v));
    }

    public static void main(String[] args) {
        String str = "Java is fun, Java is powerful.";
        FrequencyCount(str);
    }
}

```

25. Check Sudoku Row Validity

```

import java.util.HashSet;

public class SudokoRowValdity {

    public static boolean checkValidity(int[] a) {

```

```

int n = a.length;
HashSet<Integer> set = new HashSet<>();

for (int i = 0; i < a.length; i++) {

    if ((a[i] <= 0 || a[i] > 9) || set.contains(a[i])) {
        return false;
    }
    set.add(a[i]);
}

if (set.size() == 9)
    return true;
else
    return false;
}

public static void main(String[] args) {
    int[] a = { 5, 3, 4, 6, 7, 8, 9, 1, 2 };
    System.out.println(checkValidity(a));

    int[] b = { 5, 3, 4, 6, 7, 8, 9, 1, 1 };
    System.out.println(checkValidity(b));

    int[] c = { 5, 3, 4, 6, 7, 8, 9, 1, 10 };
    System.out.println(checkValidity(c));
}
}

```

26.Spiral Matrix Generator

```

public class SpiralTraversal {

    public static void SpiralMatrix(int n) {
        int top = 0, left = 0, right = n - 1, bottom = n - 1;
        int[][] matrix = new int[n][n];
        int no = 1;

        while (top <= bottom && left <= right) {

            for (int i = left; i <= right; i++) {

```

```

        matrix[top][i] = no;
        no++;
    }
    top++;

    for (int i = top; i <= bottom; i++) {
        matrix[i][right] = no;
        no++;
    }
    right--;

    if (top <= bottom) {
        for (int i = right; i >= left; i--) {
            matrix[bottom][i] = no;
            no++;
        }
        bottom--;
    }

    if (left <= right) {
        for (int i = bottom; i >= top; i--) {
            matrix[i][left] = no;
            no++;
        }
        left++;
    }
}

for (int i = 0; i < n; i++) {
    for (int j = 0; j < n; j++) {
        System.out.print(matrix[i][j] + " ");
    }
    System.out.println();
}

}

public static void main(String[] args) {
    SpiralMatrix(4);
}
}

```

27. Password Strength Validator

```
public class PasswordChecker {

    public static boolean isValidPassword(String str) {

        boolean isUpperCasePresent = false;
        boolean isLowerCasePresent = false;
        boolean isDigitPresent = false;
        boolean isSpecialCharacterPresent = false;
        String specialCharacters = "!@#%$%^&*()+";

        if (str.length() < 8)
            return false;

        for (int i = 0; i < str.length(); i++) {
            char ch = str.charAt(i);

            if (Character.isUpperCase(ch))
                isUpperCasePresent = true;
            else if (Character.isLowerCase(ch))
                isLowerCasePresent = true;
            else if (Character.isDigit(ch))
                isDigitPresent = true;
            else if (specialCharacters.contains(ch + ""))
                isSpecialCharacterPresent = true;
        }

        return isUpperCasePresent && isLowerCasePresent && isSpecialCharacterPresent &&
isDigitPresent;

    }

    public static void main(String[] args) {

        System.out.println(isValidPassword("Hello+123")); // true
        System.out.println(isValidPassword("Hello123")); // false (no special character)
        System.out.println(isValidPassword("hello+123")); // false (no uppercase)
        System.out.println(isValidPassword("HELLO+123")); // false (no lowercase)
        System.out.println(isValidPassword("Hello+12")); // true

    }
}
```



```
}
```

28. Number Pyramid Pattern

```
public class NumberPyramidPattern {  
  
    public static void Pyrimid(int n) {  
  
        int no = 1;  
  
        for (int i = 1; i <= n; i++) {  
  
            for (int j = i; j < n; j++) {  
                System.out.print(" ");  
            }  
  
            for (int k = 1; k <= i; k++) {  
                System.out.print(no + " ");  
                no++;  
            }  
            System.out.println();  
        }  
    }  
  
    public static void main(String[] args) {  
        int n = 4;  
        Pyrimid(n);  
    }  
}
```

29. Digit Frequency Counter

```
import java.util.HashMap;  
import java.util.Scanner;  
  
public class DigitFrequencyCounter {  
    public static void main(String[] args) {  
        int no;  
  
        System.out.println("Enter the Question");
```

```

Scanner in = new Scanner(System.in);
no = in.nextInt();

String str = new String(no + "");

HashMap<Character, Integer> map = new HashMap<>();

for (int i = 0; i < str.length(); i++) {
    char ch = str.charAt(i);
    map.put(ch, map.getOrDefault(ch, 0) + 1);
}

map.forEach((k, v) -> System.out.println(k + " " + v));
}
}

```

30. Magic Square Validator

```

public class MagicSquare {

    public static boolean isMagicSquare(int[][] a) {
        int rows = a.length;
        int column = a[0].length;
        int initialSum = 0;

        for (int i = 0; i < rows; i++) {
            initialSum += a[0][i];
        }

        for (int i = 1; i < rows; i++) {
            int rowsum = 0;

            for (int j = 0; j < column; j++) {
                rowsum += a[i][j];
            }

            if (initialSum != rowsum)
                return false;
        }
    }
}

```

```

for (int i = 0; i < rows; i++) {

    int colsum = 0;

    for (int j = 0; j < column; j++) {
        colsum += a[j][i];
    }

    if (colsum != initialSum)
        return false;
}

// First Diagonal
int diagonalSum1 = 0;

for (int i = 0; i < rows; i++) {
    diagonalSum1 += a[i][i];
}
if (diagonalSum1 != initialSum)
    return false;

// Second Diagonal
int diagonalSum2 = 0;

for (int i = 0; i < rows; i++) {
    diagonalSum2 += a[i][a.length - i - 1];
}

if (diagonalSum2 != initialSum)
    return false;

return true;
}

public static void main(String[] args) {

    int[][] matrix = {
        { 1, 2, 3 },
        { 4, 5, 6 },
        { 7, 8, 9 }
    };

    int[][] matrix1 = {

```

```
        { 2, 7, 6 },
        { 9, 5, 1 },
        { 4, 3, 8 }
    };

    if (isMagicSquare(matrix1)) {
        System.out.println("Magic Square...");
    } else {
        System.out.println("Not Magic Square...");
    }
}
}
---- End of First PDF ----
```